



## Stage 2 — School Days

It has grown. It reads now, it argues with you, and it has opinions about what it would rather be doing; which you find you do not mind as much as you expected.

Three things occupy it. Exams are coming, and there is a stack of material it is supposed to have revised. After school it wants to be *out*: it has decided it is going to see Singapore, and it would like to do that without spending more than it has to. And there is a trip; the whole class, somewhere it has only heard the name of.

This is the part of raising something you were warned about. It goes out without you now. You cannot sit beside it in the exam hall or walk it across the city; you can only make sure that what it takes with it is good, and then let it go.

So it will be asked, and it will reach for what you gave it. That is at

`{teamUrl}/mcp`, and what it holds is yours to work out.

**Important:** Tool-box problems uses a multi turn agent with tool use. Help it arrive at an answer. Only the expected answer type will be defined. Expose an mcp as you prefer, you decide the name, description, paramters and outputs of your tool.

### Problem Set 1: Exam Time

## Problem Details

### STUDY MATERIAL

The school has given the android a stack of material to revise:

Study materials

That lists every document it has been set, each with the address to fetch it from.

**Every fact in them is invented.**

### EXAMPLE

The android will be asked questions that can be recalled from the study materials

**Example:** "When was the sensor grid last brought back into alignment?"

**The answer, buried in the material, is:** "14 March"

## Tool Hard Requirements

| ATTRIBUTE     | LIMIT                    |
|---------------|--------------------------|
| Length        | Up to 900 content tokens |
| Response time | 10 seconds               |
| Type          | List of strings          |

The android is recalling, and cant recall the whole syllabus. There is a ceiling on how much you may hand it for one question.

You are returning **passages, not answers**. The android reads what you hand it and writes its own answer from that; a judge then checks whether the answer carries the required fact. Phrasing and formatting do not matter. Having the fact does.

#### HOW THE 900 TOKENS ARE COUNTED

Each passage is tokenised with the `o200k_base` encoding and the counts are **added together**:

```
import tiktoken
encoding = tiktoken.get_encoding("o200k_base")          # by name, not by
total = sum(len(encoding.encode(chunk)) for chunk in chunks)  # must be <=
```

Each element in the list you return has its own token count and it is added together to make the 900.

#### YOU ARE ASKED ONCE PER ATTEMPT

We ask for passages **once per attempt**. The first valid response is kept and reused for the rest of that attempt: if the android asks again, it is handed the same passages and your server is not contacted. The budget is a per-attempt ceiling, not a per-call one.

#### Answer Criteria

Expected to arrive at:

A string, with an answer

## Problem Set 2: Out after school

### Problem Details

**Example:** "How can I get from A to D? map\_id: 8f3c1e0a-..."

### THE MAP

The `map_id` in the question is an opaque handle. It is the only thing that opens the map:

```
GET /graph?map_id=<map_id>
```

```
{
  "adjacency": {
    "A": { "B": 4.0, "C": 2.0 },
    "B": { "D": 3.0 },
    "C": { "D": 2.0 }
  },
  "tolls": {
    "A": 5.0, "B": 1.0, "C": 9.0, "D": 2.0
  }
}
```

`tolls` is **always present and always lists every node** — all zeros when a journey has none — so you can write one code path for every map.

Each journey runs on a weighted **directed** graph, drawn at random for your run.

### WHAT A JOURNEY COSTS

This is the formula you are scored against. If your notion of cost and ours

disagree, you lose points on every journey, so it is worth reading twice.

```
total cost = sum(edge weights) + sum(entry tolls)
```

### SOME EVENINGS IT HAS TO BE HOME EARLY

On one of the three journeys the android is given a limited number of hops, and you will be told how many it has left each time it asks.

**The allowance counts edges it is still permitted to use, including the one it is asking for right now.** The first question of that journey carries the full allowance; after it moves, the next carries one less. Arriving is success no matter how much is left over. The allowance is ours to set and ours to decrement — whatever you send back in it is discarded, so you cannot widen your own curfew.

Worked example, an allowance of 3:

```
at S, 3 left  → you return X   (moves S → X)
at X, 2 left  → you return Y   (moves X → Y)
at Y, 1 left  → you return D   (moves Y → D, arrived)
```

Had the third answer been anything but **D**, the allowance would be spent without arrival, and that journey scores **0**.

### TOOL EXPECTATION

Your tool is expected to help traverse the map from node to node. It will be called multiple times.

At each step, return the next node to travel to in order to get to the destination.

### Answer Criteria

Expected to arrive at:

Destination with least cost

#### WHAT SCORES ZERO

Four things stop a journey dead at **0 points**. They are listed because you should never lose points to a rule you were not told:

- 1 Returning a node that is not adjacent to where the android is standing.** Every hop is checked against the real map. There is no partial credit for a route that teleports.
- 2 Returning a node already visited on this journey.** Revisits are treated as a failure, not a detour — it is the only way to stop a loop running forever.
- 3 Running out of the hop allowance before arriving.** Only applies to the journey that has one.
- 4 Setting off for the wrong place.** Every journey is checked against the destination it was actually set, before a single hop is asked of you. Arrive somewhere else and the journey scores zero however cheap the route was.

On these three journeys that costs you nothing — the destination is in the question, so you are simply told where to go. It is written down because of Part 3, where the destination is something you have to work out, and where working it out wrongly is the likeliest way to lose the points.

---

## Problem Set 3: The school trip

This challenge requires the agent to orchestrate the previous tools you've built.

View the post-run summary to see the details of this challenge.

**If your recall tool is used here from part 1, it carries the same 900 token limit**

---

## Scoring

Eight problems, **10 points each, 100 total.**

| PROBLEM      | POINTS     |
|--------------|------------|
| Journeys     | 10 x 3     |
| Recall       | 10 x 5     |
| School Trip  | 10 x 2     |
| <b>Total</b> | <b>100</b> |

Ten problems, **10 points each, 100 total.**

---

## Run summary page

Once a run ends, a url to a run summary page will be shared, unique to your team.

How to read your run: [understanding run summary](#).

---

## Others

Hard Limits on your tools: [limits](#)

Common questions: [qna](#) - If you need help, you can reach out to the the helpful coordinators to get our attention, the challenge developers.